Agentic AI Interview Questions: Detailed Answers

A thinking-and-reasoning guide to agentic AI interviews. Each question has a **model answer**, the **reasoning behind it**, a **concrete example**, and a likely **interviewer follow-up**.

1. What makes an AI system "agentic" rather than a traditional chatbot?

Model answer: A chatbot maps one input to one output. An agentic system pursues a goal over multiple steps, deciding for itself what to do next, acting through tools, and adjusting based on what happens.

Detailed reasoning:

- **Goal orientation:** The unit of work is a goal ("get me to the Singapore conference within budget"), not a single prompt.
- **Planning and decomposition:** It breaks the goal into subtasks and orders them by dependency.
- **Tool use:** It changes the world or fetches live information (APIs, databases, code execution, browsers) rather than only generating text.
- **The control loop:** It runs perceive, reason, act, observe, repeat. The loop is the real architectural difference. The model's output at step N becomes input to step N+1.
- **Evaluation and adaptation:** It checks whether an action worked and changes strategy when it didn't (a failed API call, an empty search result).
- **Asking only when needed:** It resolves ambiguity itself where it safely can and asks the user only for genuinely missing information.

Example: Asked "How do I book a vacation?", a chatbot explains the process. An agent asks for dates and budget, searches flights and hotels, compares options, drafts an itinerary, and requests approval before booking.

Nuance: Agency is a spectrum, not a switch. A chatbot with one search tool is slightly agentic. A system that plans, executes, and self-corrects for hours is highly agentic.

Follow-up: "Is a RAG chatbot agentic?" Usually not. Retrieval then generation is a fixed pipeline. It becomes agentic when the model decides whether to retrieve, what to retrieve, and whether the results are good enough.

2. If an agent fails midway through a 20-step workflow, how would you design recovery?

Model answer: Treat the agent like a distributed system: make state explicit and durable, make steps idempotent, and resume from the last verified checkpoint rather than restarting.

Detailed design:

1. **Persist state outside the LLM context.** After every step, store the step ID, inputs, outputs, status (pending, running, succeeded, failed), timestamps, and the plan version. A database or workflow engine holds it, not the conversation history.
2. **Checkpoint at meaningful boundaries.** Checkpoint after each successful tool call and after any irreversible action. Frameworks like LangGraph provide built-in checkpointers, and engines like Temporal give durable execution.
3. **Make steps idempotent.** If step 14 ("send invoice") crashed after sending but before recording success, a naive retry sends the invoice twice. Use idempotency keys, or check the real system state before re-executing ("does invoice #123 already exist?").
4. **Classify the failure:**
   - *Transient* (timeout, rate limit): retry with exponential backoff and jitter.
   - *Recoverable by changing approach* (bad parameter, wrong tool): feed the error back to the agent so it can replan.
   - *Permanent* (permission denied, missing data): escalate to a human.
5. **Compensating actions for rollback.** Not everything can be undone. For steps with side effects, define compensations (the saga pattern): cancel the booking, refund the charge, delete the draft. Unwind in reverse order.
6. **Resume with rehydration.** On restart, rebuild the agent's context from stored state: the goal, the completed steps with summarized results, and the remaining plan. Don't replay the whole transcript.
7. **Plan validity check.** After a long pause the world may have changed (the flight price moved). Re-validate assumptions before continuing.

Example: The agent fails at step 13 of 20 while booking a hotel. Steps 1 to 12 are checkpointed, so it resumes at 13. The flight booked at step 9 stays booked, and if the hotel can't be secured, a compensation step cancels the flight.

Follow-up: "What if the checkpoint itself is stale or wrong?" Verify against the source of truth before resuming, and log checkpoint versions so you can audit and roll back.

3. How would you prevent an LLM agent from entering an infinite reasoning loop?

Model answer: Layer several defenses: hard limits, progress detection, and an escalation path. No single safeguard is enough, because loops come in different shapes.

Types of loops:

- *Exact repetition:* the same tool call with the same arguments repeatedly.
- *Oscillation:* A, B, A, B.
- *Spinning:* varied calls that make no real progress.
- *Over-deliberation:* endless planning and never acting.

Safeguards:

Layer	Mechanism	Purpose
Hard limits	Max steps, tokens, wall-clock time, spend	Absolute backstop
Repetition detection	Hash recent (tool, args) pairs; flag duplicates	Catches exact loops
Progress metrics	Is state changing? New information gained? Subgoals completed?	Catches spinning
Completion check	Verify the goal with an independent check, not the agent's own claim	Prevents premature or never-ending termination
Forced replan	After N failures of the same kind, require a different strategy	Breaks out of ruts
Escalation	Hand off to a human or return a partial result with explanation	Graceful exit

```
MAX_STEPS = 10
seen = {}

for step in range(MAX_STEPS):
    action = agent.decide(state)
    key = hash((action.tool, json.dumps(action.args, sort_keys=True)))
    seen[key] = seen.get(key, 0) + 1

    if seen[key] >= 3:
        state = agent.force_replan(state, reason="repeated action")
        continue

    result = execute(action)
    state = update(state, result)

    if verify_goal(state):          # independent verifier
        return success(state)

return escalate(state, reason="step budget exhausted")
```

Important nuance: When the limit is hit, don't just crash. Return what was accomplished, what blocked progress, and a recommendation. A partial answer with an explanation is far more useful than a timeout.

Follow-up: "Why not just set a very low max-steps?" Too low and legitimate long tasks fail. Set limits per task type based on measured step distributions, and combine them with progress detection so legitimate work isn't cut off.

4. An agent uses RAG and tool calling. Which should happen first?

Model answer: Neither has a fixed order. The agent decides based on what information is missing. The principle: **gather the knowledge you need before you act, and act in order to gather knowledge you can't retrieve.**

Reasoning process:

- If the task needs *stored knowledge* (policies, documents, history), retrieve first.
- If the task needs *live or computed data* (current price, calculation, system state), call a tool first.
- If the task needs to *change something*, retrieve the rules first (what's allowed), then act.
- Complex tasks interleave: retrieve, call a tool, retrieve again based on what the tool returned.

Why retrieval before action matters: Acting without relevant policy or context is how agents make confident, wrong, sometimes costly moves. Retrieving the refund policy before issuing a refund is cheaper than reversing a bad refund.

Subtle point: In modern designs, RAG is often just another tool (`search_knowledge_base`). The agent then chooses when to call it, rewrites queries, retrieves again if results are poor, and skips it when the question is trivial. That is "agentic RAG," as opposed to a fixed retrieve-then-generate pipeline.

Example: "Create a report using yesterday's sales."

1. Tool: query the sales database for yesterday's data (live data, so a tool).
2. RAG: retrieve the report template and formatting guidelines.
3. Analyze, generate, and validate.

Both are needed here, and the order between them is flexible.

Follow-up: "What if retrieval returns nothing relevant?" The agent should detect low relevance, reformulate the query, try another source, and if still nothing, say so rather than guessing.

5. How would you evaluate whether an autonomous agent is performing well?

Model answer: Evaluate outcome, trajectory, and operational health. Accuracy alone misses most agent failures.

1. Outcome metrics (did it work?)

- Task success rate, measured against a verifiable end state wherever possible (the record exists, the tests pass), not the agent's self-report.
- Partial-credit scoring for multi-part tasks.

2. Trajectory metrics (how did it work?)

- Correct tool selection and valid arguments
- Number of steps versus the optimal path
- Retries and error-recovery quality
- Unsafe or out-of-scope actions attempted, even if blocked

3. Operational metrics

- Cost per task and per successful task
- Latency (end to end and per step)
- Human intervention and escalation rate

- Tool success rate

4. Experience and trust metrics

- User satisfaction, corrections, and override rate
- Intent alignment (see the bonus question)

Methods:

- **Golden test suites** of realistic tasks with known outcomes, including edge cases and adversarial ones.
- **Repeated runs.** Agents are non-deterministic, so report pass rates over many runs (pass@k and pass^k, since reliability means succeeding every time, not once).
- **LLM-as-judge** with explicit rubrics, calibrated against human labels.
- **Production sampling** with human review, plus **regression tests** built from every real failure.
- **Online monitoring** and A/B tests when changing prompts or models.

Follow-up: "Your success rate is 85%. Is that good?" It depends on the cost of failure and what the other 15% looks like. Ten percent harmless misses are very different from 1% catastrophic ones. Segment failures by severity.

6. Why might a smaller LLM sometimes be better than GPT-4-class models in an agent?

Model answer: Because agent systems are bounded by cost, latency, and reliability across many calls, and most steps don't need frontier reasoning.

Detailed reasons:

- **Compounding latency and cost:** An agent may make 20 to 50 model calls per task. Small per-call savings multiply.
- **Narrow tasks suit small models:** Classification, routing, extraction, formatting, and summarization are often handled well by small or fine-tuned models.
- **Predictability:** A fine-tuned small model on a narrow task can be more consistent than a general model.
- **Deployment constraints:** On-device, privacy-sensitive, or air-gapped environments may require small models.
- **Throughput:** Faster iterations mean more attempts, more verification, and better UX.
- **Redundancy:** Cheap calls let you sample several times and vote.

Pattern: model routing / tiering

```
Router (small)         -> classify the request, choose a path
Executors (small/mid)  -> extract fields, call routine tools, format output
Planner/solver (large) -> ambiguous reasoning, hard decisions, final review
Verifier (mid/large)   -> check high-stakes outputs
```

A common shortcut is to give planning to the small model. That is debatable, because planning is often where capability matters most. A more defensible split is a strong model for planning and hard reasoning, and smaller models for executing well-defined steps.

Caveat: Don't over-optimize early. Start with a strong model to establish quality, then measure where cheaper models match it.

7. Explain the difference between planning and reasoning in agent systems.

Model answer: Reasoning is inference: drawing conclusions from information, such as diagnosing, comparing, and deciding. Planning is structuring future action: choosing and sequencing steps to reach a goal.

	Reasoning	Planning
Question answered	"What is true / what should I conclude?"	"What sequence of actions gets me there?"
Output	A conclusion or decision	An ordered, dependency-aware set of steps
Time horizon	Immediate	Multi-step future
Failure	Wrong inference	Wrong order, missing step, infeasible plan

They interact constantly. Planning requires reasoning (is this step feasible? what does it depend on?), and reasoning about a result often triggers re-planning. ReAct interleaves them: think, act, observe, think. Plan-and-execute separates them: plan first, then execute with local reasoning.

Example: "Will my flight be on time?"

- *Reasoning:* "To answer this I need live flight status and weather."
- *Planning:* (1) look up the flight number, (2) call the status API, (3) check the weather at the departure airport, (4) synthesize and respond.

The "what vs. how" shorthand is a useful intuition but oversimplified, since reasoning also determines how.

Follow-up: "When would you separate them architecturally?" For long, costly, or high-stakes tasks where a reviewable plan, human approval of the plan, and cheaper executors are valuable.

8. How would you design a multi-agent system for software development?

Model answer: Use an orchestrator-worker structure with specialized agents, a shared state or workspace, objective verification gates, and human approval at key points.

Roles:

- **Orchestrator/Planner:** turns requirements into tasks, assigns work, tracks status, and is the single accountable owner.
- **Coding agents:** implement scoped tasks in isolated branches or sandboxes.
- **Test agent:** writes and runs unit and integration tests.
- **Security agent:** static analysis, dependency and secret scanning, threat review.
- **Reviewer agent:** checks quality, style, and requirements coverage.
- **Optional:** docs agent, DevOps agent, requirements/product agent.

Design considerations:

1. **Shared workspace, not shared chat.** Agents coordinate through artifacts (repo, tickets, specs, test reports), not by reading each other's entire transcripts.
2. **Precise task briefs.** Each delegation includes the objective, constraints, interfaces, acceptance criteria, and what not to touch. Vague delegation is the top failure cause.
3. **Parallelism with isolation.** Independent modules run in parallel on separate branches, with a merge step handling conflicts.
4. **Objective gates.** Tests pass, linters pass, scans pass. Don't rely only on an LLM saying "looks good."
5. **Sandboxed execution** with least-privilege credentials.
6. **Human checkpoints:** approve the plan, approve merges to main, approve deploys.
7. **Limits:** iteration caps on the code-test-fix loop.

```
Requirements -> Orchestrator/Planner
                      | (task briefs)
        +-------------+-------------+
    Coding(A)    Coding(B)     Test agent
        +-------------+-------------+
                  Security scan
                      |
                  Reviewer --fail--> back to coding with feedback
                      | pass
              Human approval -> Merge/Deploy
```

Follow-up: "Would you really use multiple agents?" Only if the task benefits. Start with one capable agent with good tools. Add agents when you need parallelism, distinct contexts, or independent verification.